

Cornell Bowers C-IS

College of Computing and Information Science

Deep Learning

Week 05: Self-Supervised Vision Networks

Thanks to:

Varsha Kishore

Justin Lovelace

Anissa Dallmann

Dylan Van Bramer

Logistics

- HW2 due today (late due date 3/5)
- A2 due today (late due date 3/5)
- Project Proposal due 3/7 (late due date 3/9)
- HW3 will be released after class, due 3/12 (late due date 3/14)
- A3 will be released after class, due 3/21 (late due date 3/23). Please start and try to get A3 done early – as it **requires** GPU – and also to leave time to study for the midterm on 3/24

Overview

- Learning image representations
- Pre-training
- Self-supervised learning
- Contrastive Loss
 - Data Augmentation
 - SimCLR

x_1

The

tastiest

fruits

are

oranges

x_2

The

best

animals

are

puppies

x_3

I

enjoy

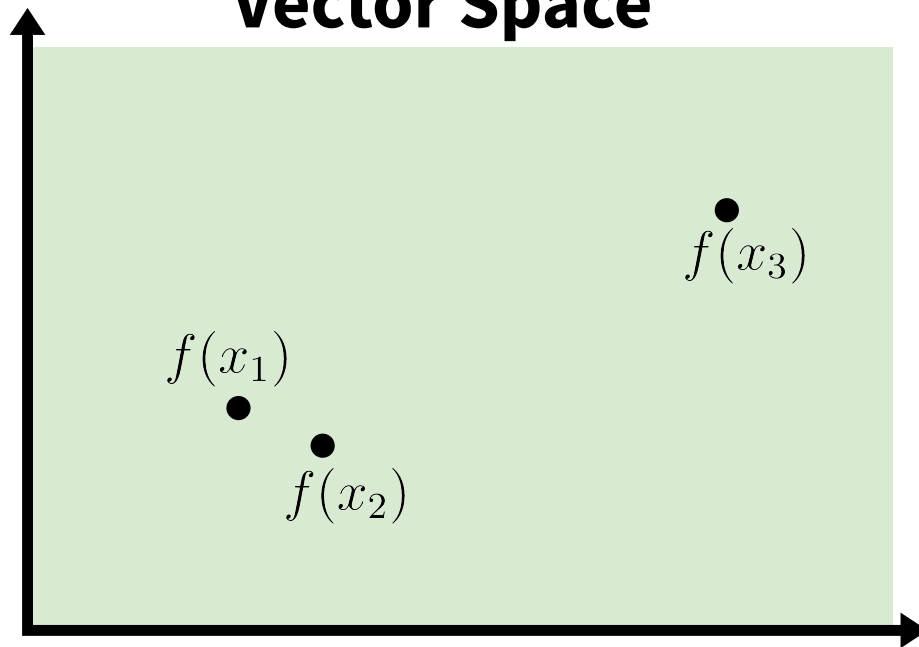
petting

baby

dogs

$f(x) = \text{bag of words}$

Vector Space



“orange”
 x_1

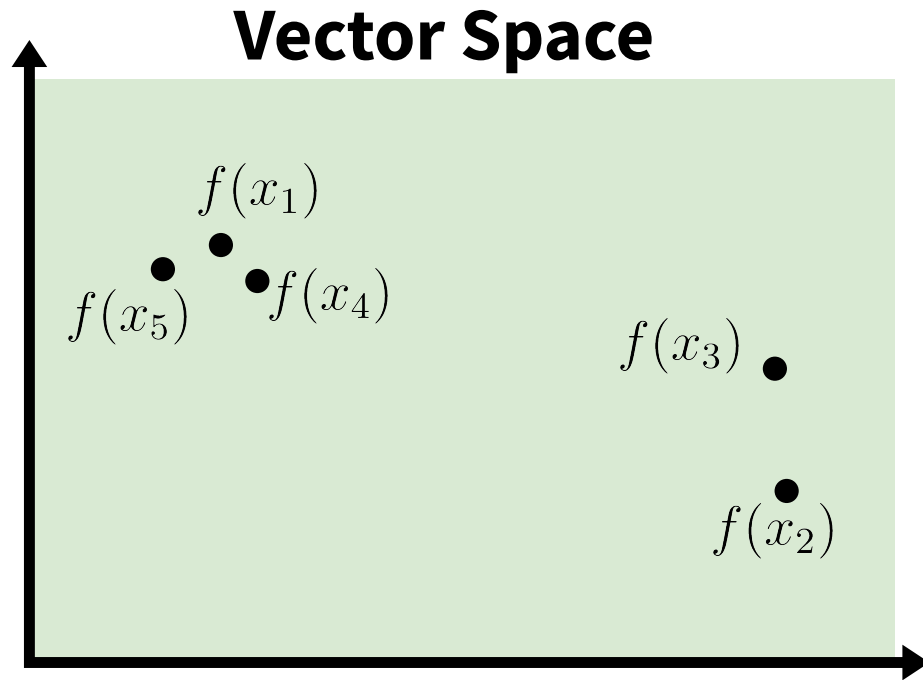
“purple”
 x_4

“puppies”
 x_2

“apple”
 x_5

“dogs”
 x_3

$f(x)$ = word embedding

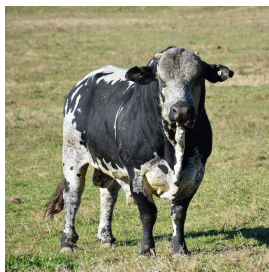


What is visual similarity?



Cornell Bowers C-IS

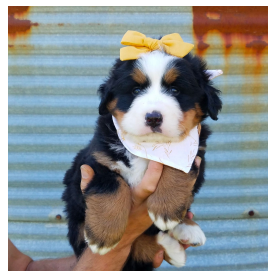
Semantically different:
puppy vs. cow



Structurally similar:
black and white
animal, grass



Structurally different:
hands, different
backgrounds



Semantically similar:
Bernese puppies

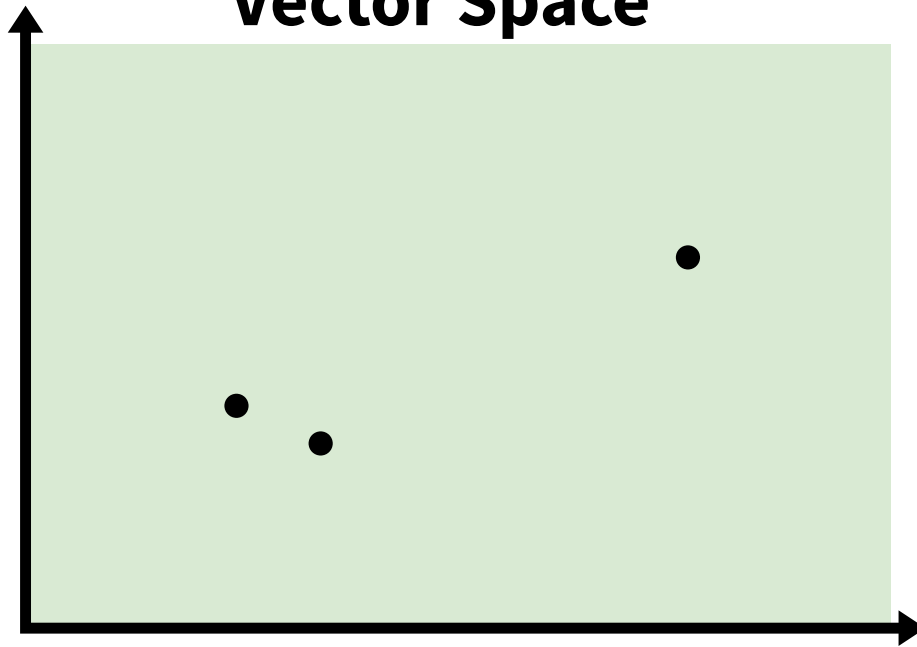
$$f(x) = ???$$

x_1

Vector Space

x_2

x_3



Pixel-Space: Nearest Neighbors

- Dominated by shallow similarities
 - Background, etc.
- Poor semantic alignment



Cifar-10 Example

(http://cs231n.stanford.edu/slides/2023/lecture_13.pdf)

$$f(x) = \text{classification network}$$

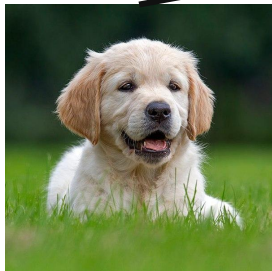
How does the network know that these should be mapped to similar space?

Class
“puppy”



x_1

Class
“puppy”



x_2

Vector Space

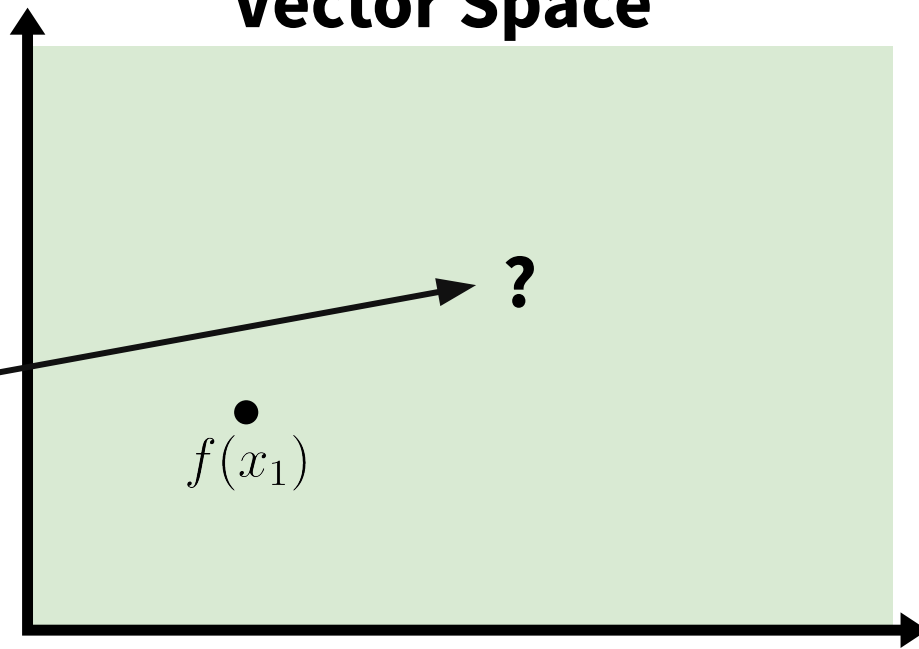
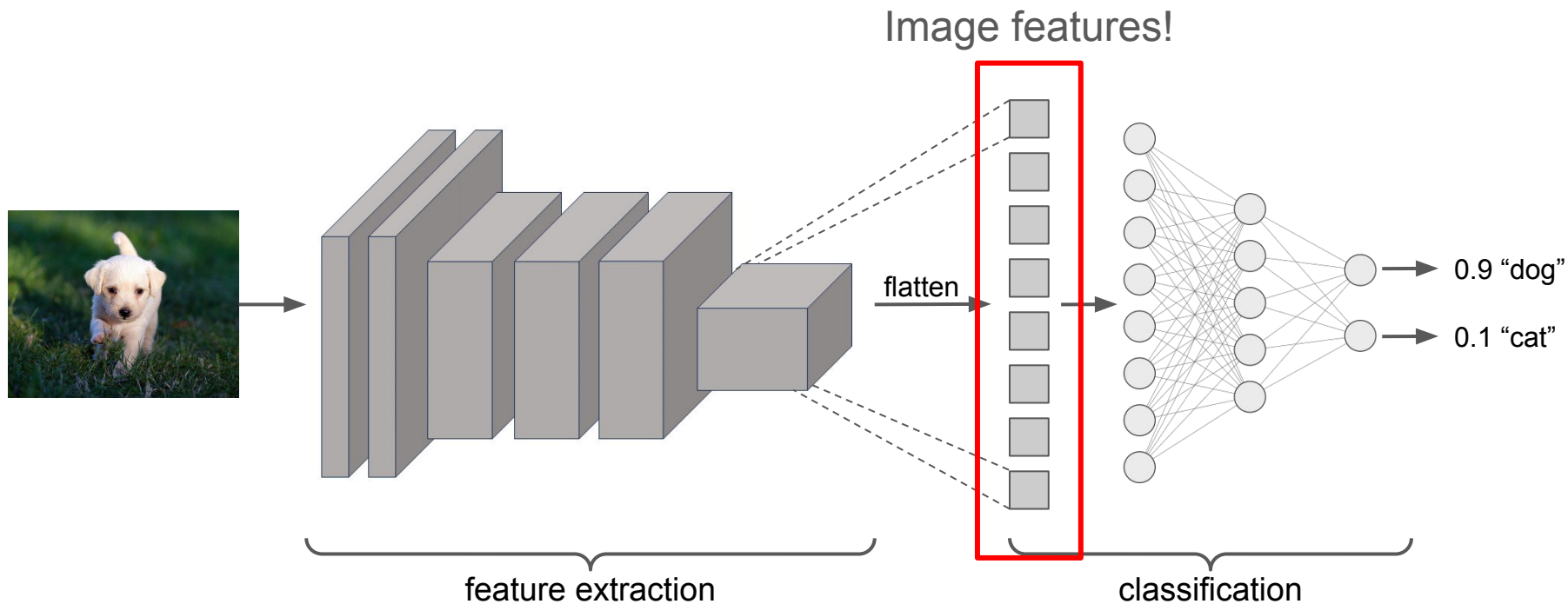


Image Classification



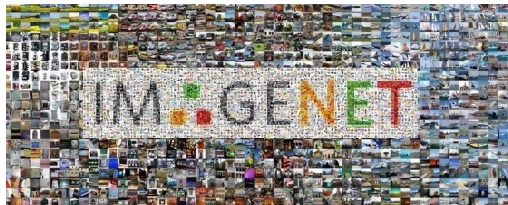
Neural Net Features: Nearest Neighbors

- Image classification features work really well!
- Strong semantic alignment
- More robust to shallow variations



ImageNet Classification with Deep Convolutional Neural Networks
by Krizhevsky et al.

Pretraining: Train a general purpose model on lots of data, and later customize it for more specific tasks



CV:
Imagenet

NLP: **Wikipedia**



Vector Space



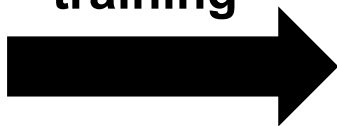
Already have a very
well-defined vector space

Image Pretraining

First, train on a large, diverse dataset so that our model learns to extract robust image features



training

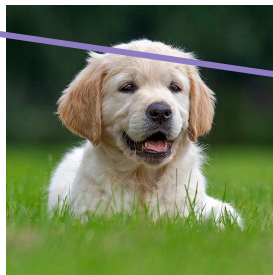


Vector Space

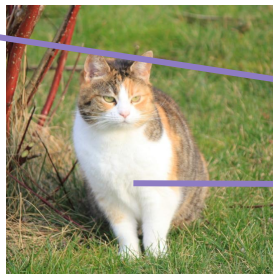


Fine-tuning

Then, finetune for a specific task

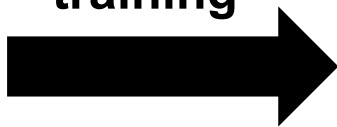


“puppy”

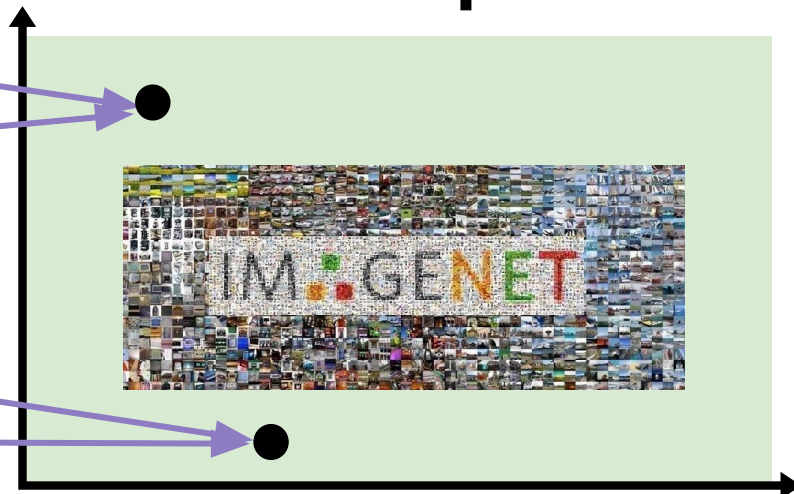


“kitten”

training



Vector Space



Pre-train



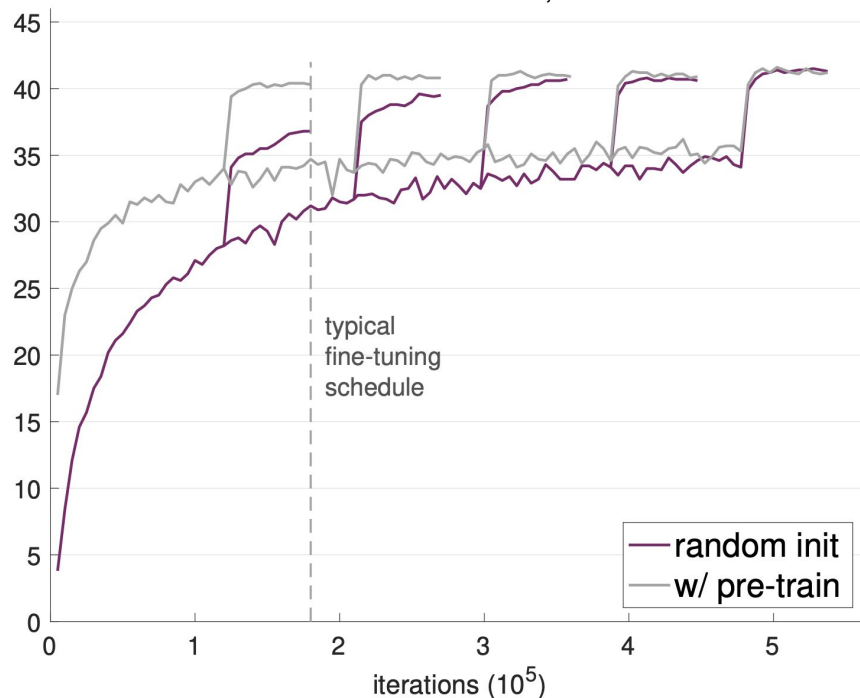
CAT

- Use image classification backbone as a feature extractor for other vision tasks
 - E.g. Detection
 - E.g. Instance segmentation
- Significantly accelerates training
 - Random init requires much longer training

Fine-tune



bbbox AP: R50-FPN, GN

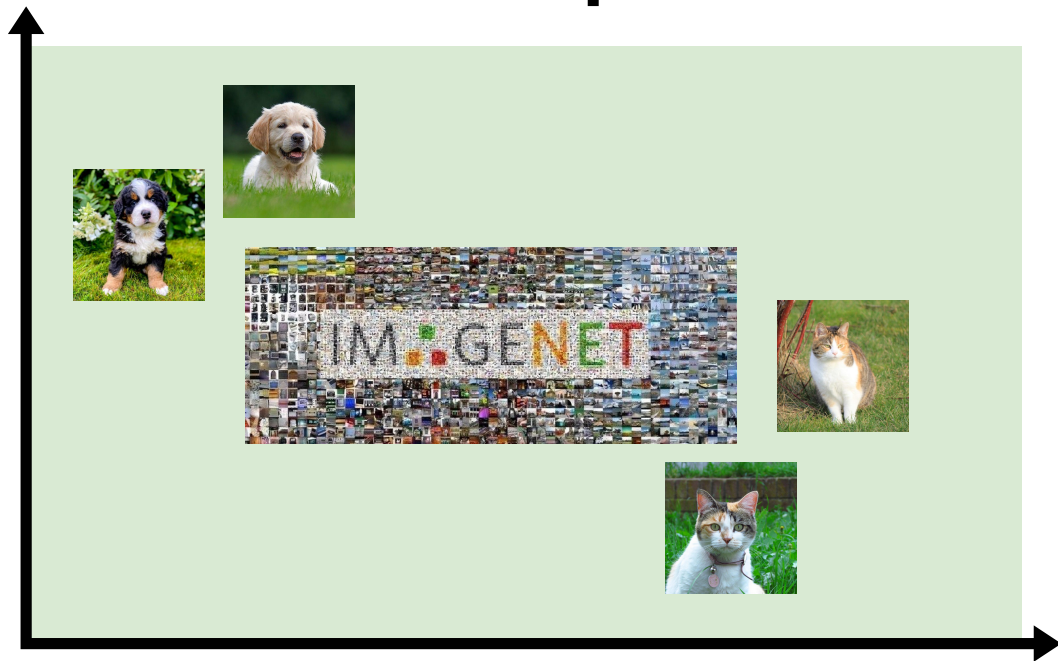


Few Shot Learning

**Adapt to variations
within known classes,
with LIMITED labeled
training data**

- We've only seen a few puppies and a few kittens, but a lot of other pretrained data

Vector Space



What are potential problems with supervised pre-training?

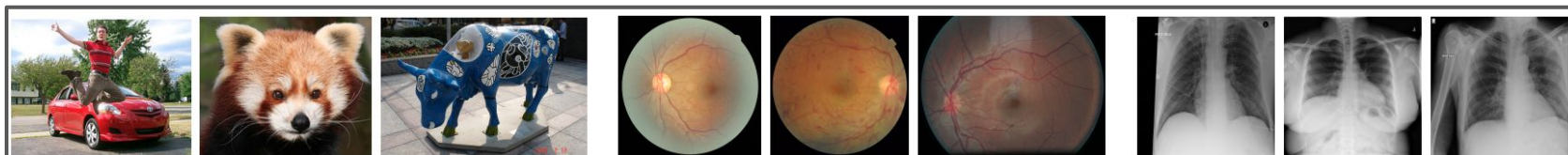
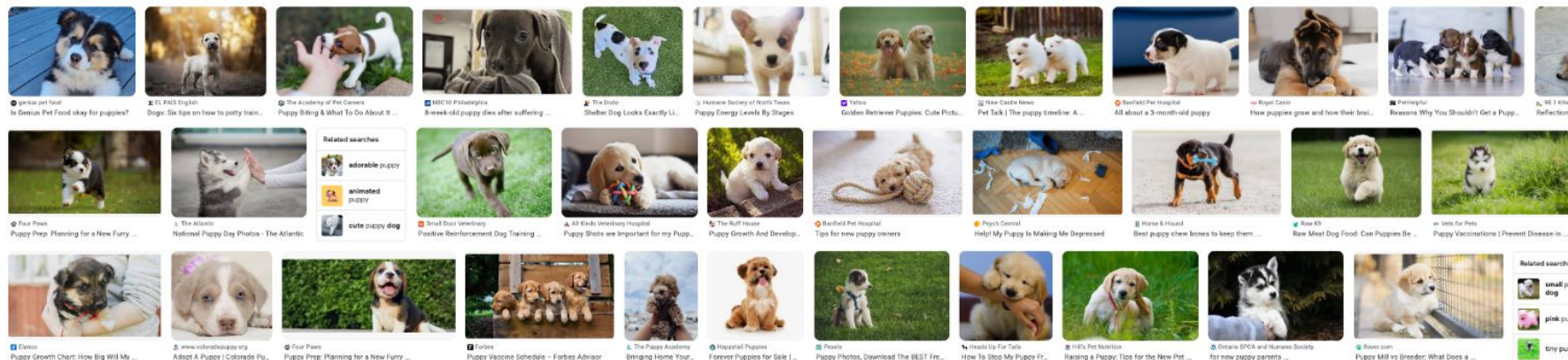


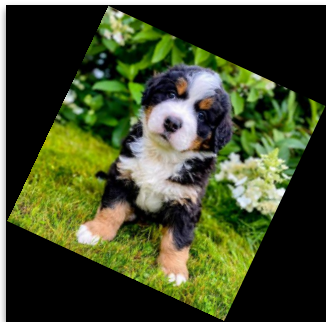
Figure 1: Example images from the IMAGENET, the *retinal fundus photographs*, and the CHEXPART datasets, respectively. The fundus photographs and chest x-rays have much higher resolution than the IMAGENET images, and are classified by looking for small local variations in tissue.

Not all images are labeled

- Particular problem for specialized domains (e.g. medicine)
 - Annotation is expensive!
- Much easier to collect unlabeled data
 - Similar to text!
- Can we still learn good image representations?



The exact same
image, rotated,
maps to a
completely
different location
in vector space



$f(x)$ = raw pixel distance

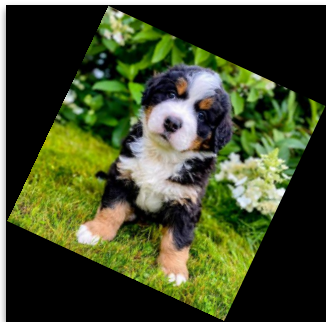
x_1

Vector Space

x_2

$f(x_1)$

$f(x_2)$



How do we learn
structure so that
these map to
similar points in
vector space?

x_1

x_2

$$f(x) = ???$$

Vector Space

$f(x_2)$
 $f(x_1)$

Self-Supervised Learning

- Aim to learn from data without manual label annotation
 - Useful for specialized domains (e.g. medicine) with limited annotated data
- Self-supervised learning methods solve “pretext” tasks that produce good features for downstream tasks.
 - Learn with supervised learning objectives (e.g., classification, regression)
 - Labels of these pretext tasks are generated automatically

Original text

Thank you ~~for~~ ~~inviting~~ me to your party ~~last~~ week.

Inputs

Thank you <X> me to your party <Y> week.

Targets

<X> for inviting <Y> last <Z>

Self-Supervised Learning

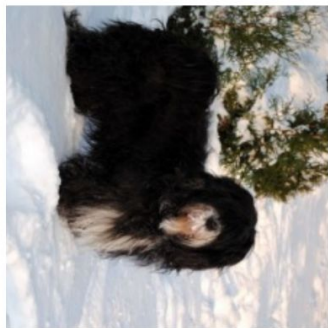


Which is the original?

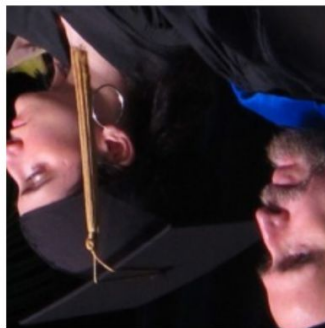
Self-Supervised Learning: Rotation Prediction



90° rotation



270° rotation



180° rotation



0° rotation



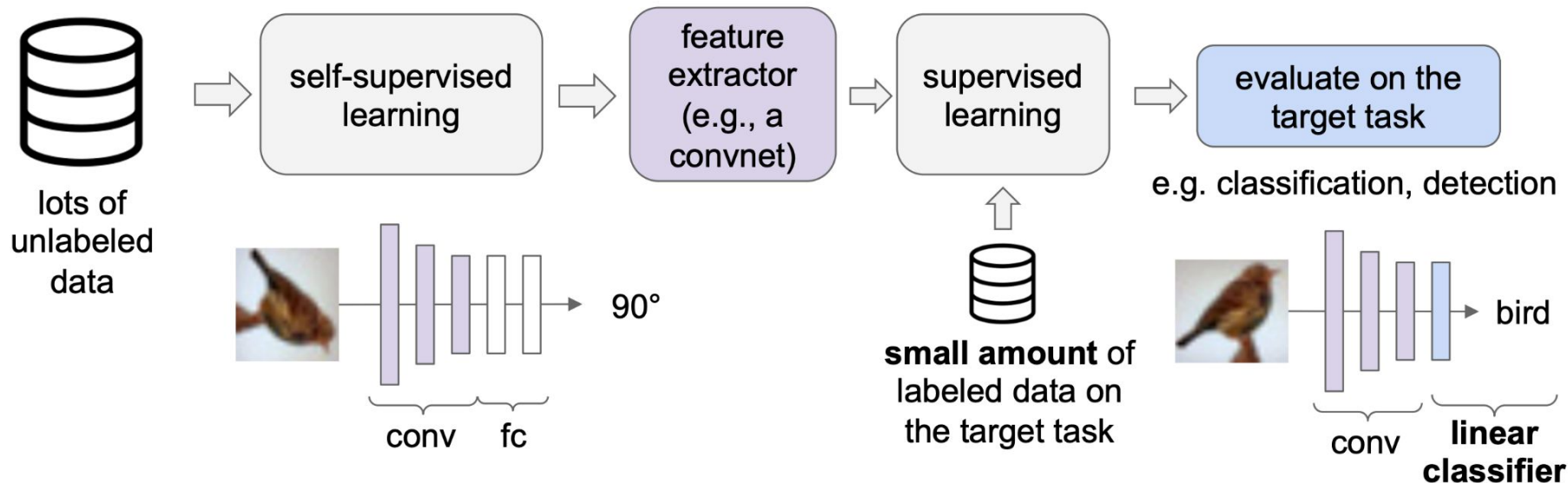
270° rotation

Figure 1: Images rotated by random multiples of 90 degrees (e.g., 0, 90, 180, or 270 degrees). The core intuition of our self-supervised feature learning approach is that if someone is not aware of the concepts of the objects depicted in the images, he cannot recognize the rotation that was applied to them.

How to evaluate a self-supervised learning method?

- Don't care about the performance of the self-supervised learning task
 - E.g. Image rotation prediction
- Evaluate the learned feature encoder on downstream target tasks

How to evaluate a self-supervised learning method?

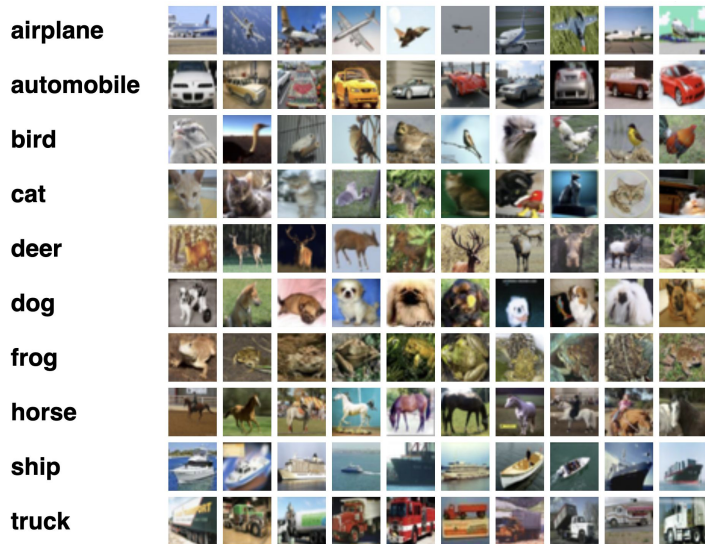


1. Learn good feature extractors from self-supervised pretext tasks, e.g., predicting image rotations

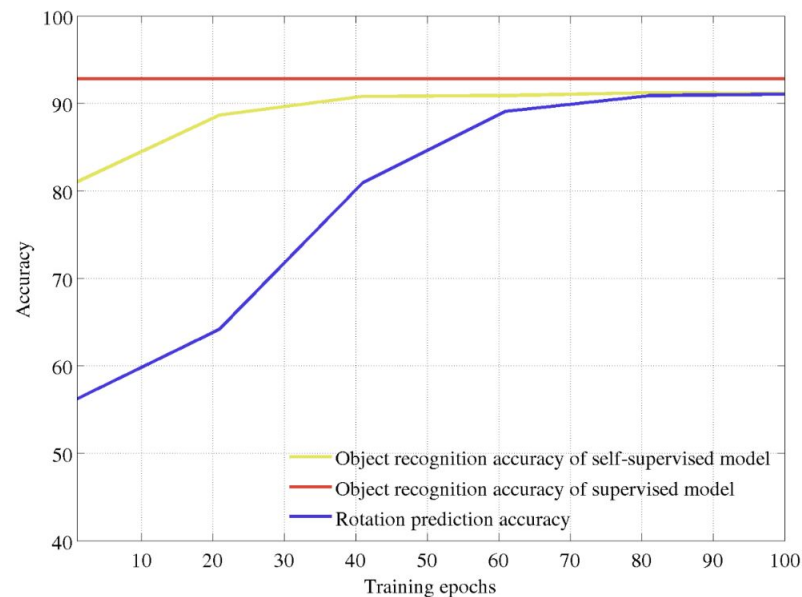
2. Attach a shallow network on the feature extractor; train the shallow network on the target task with small amount of labeled data

Self-Supervised Evaluation

- Downstream performance correlates with prefix task: rotation prediction

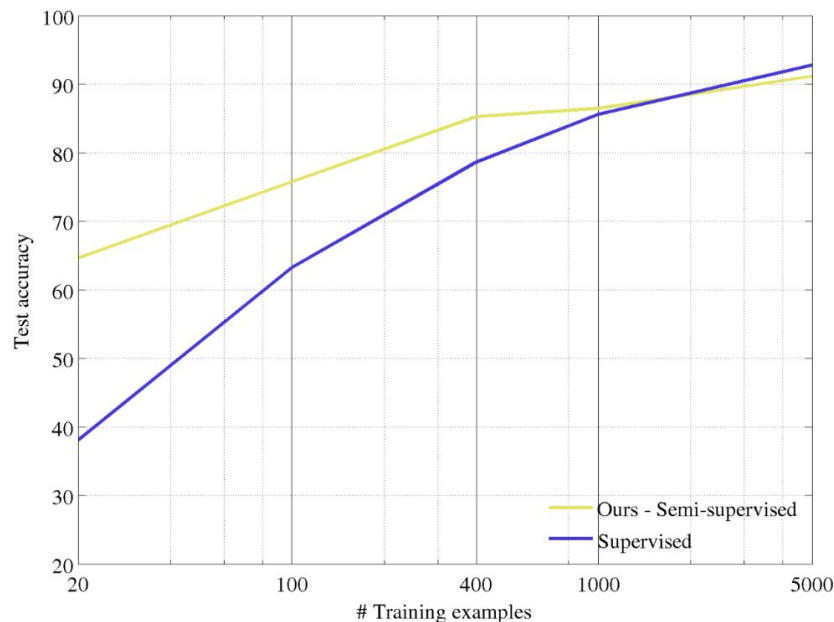


Cifar-10 Image Classification



Self-Supervised Evaluation

- Self-supervised learning outperforms supervised learning with limited data
 - Can use large volumes of unlabeled data!



Discuss

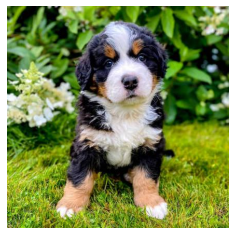
We are provided this image without labels: what are some other tasks we can do with it?

How can we perform self-supervised learning with images?

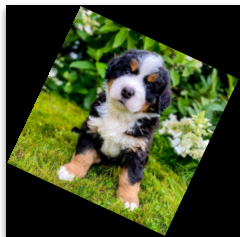


$$f(x) = ???$$

Can we learn this directly?



x_1



x_2

Vector Space

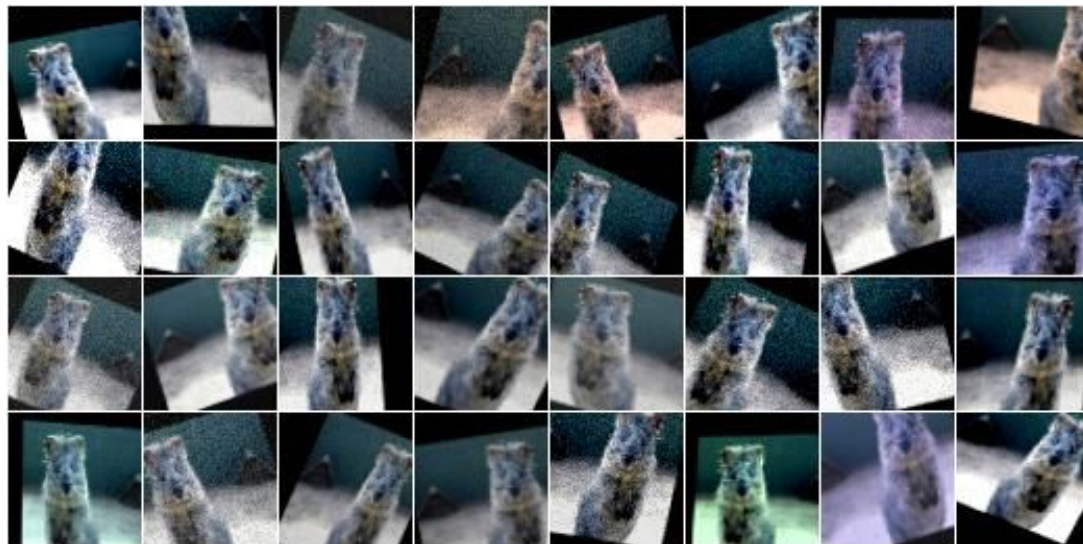
$f(x_2)$
 $f(x_1)$

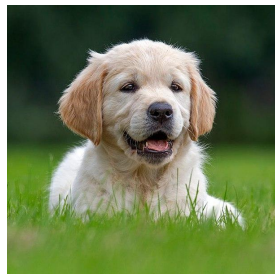
Review: Image Augmentation

- Horizontal flips
- Rotate image
- Zoom/crop image
- Brighten/darken image
- Shift colors

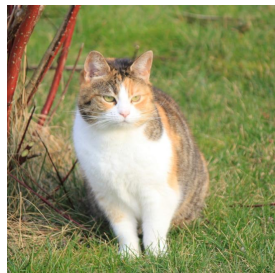


Augmentation





x_4



x_5



x_1



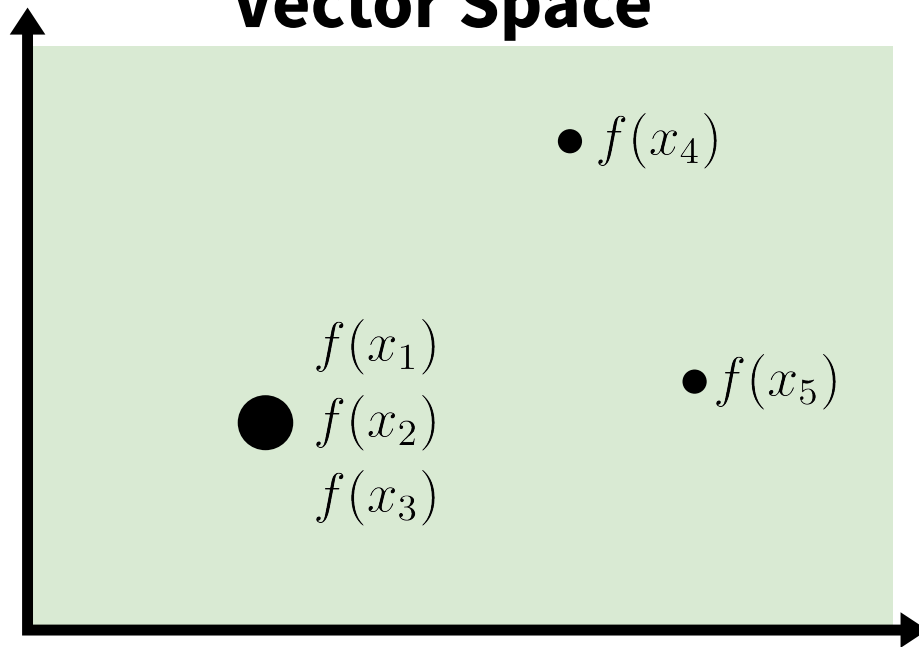
x_2



x_3

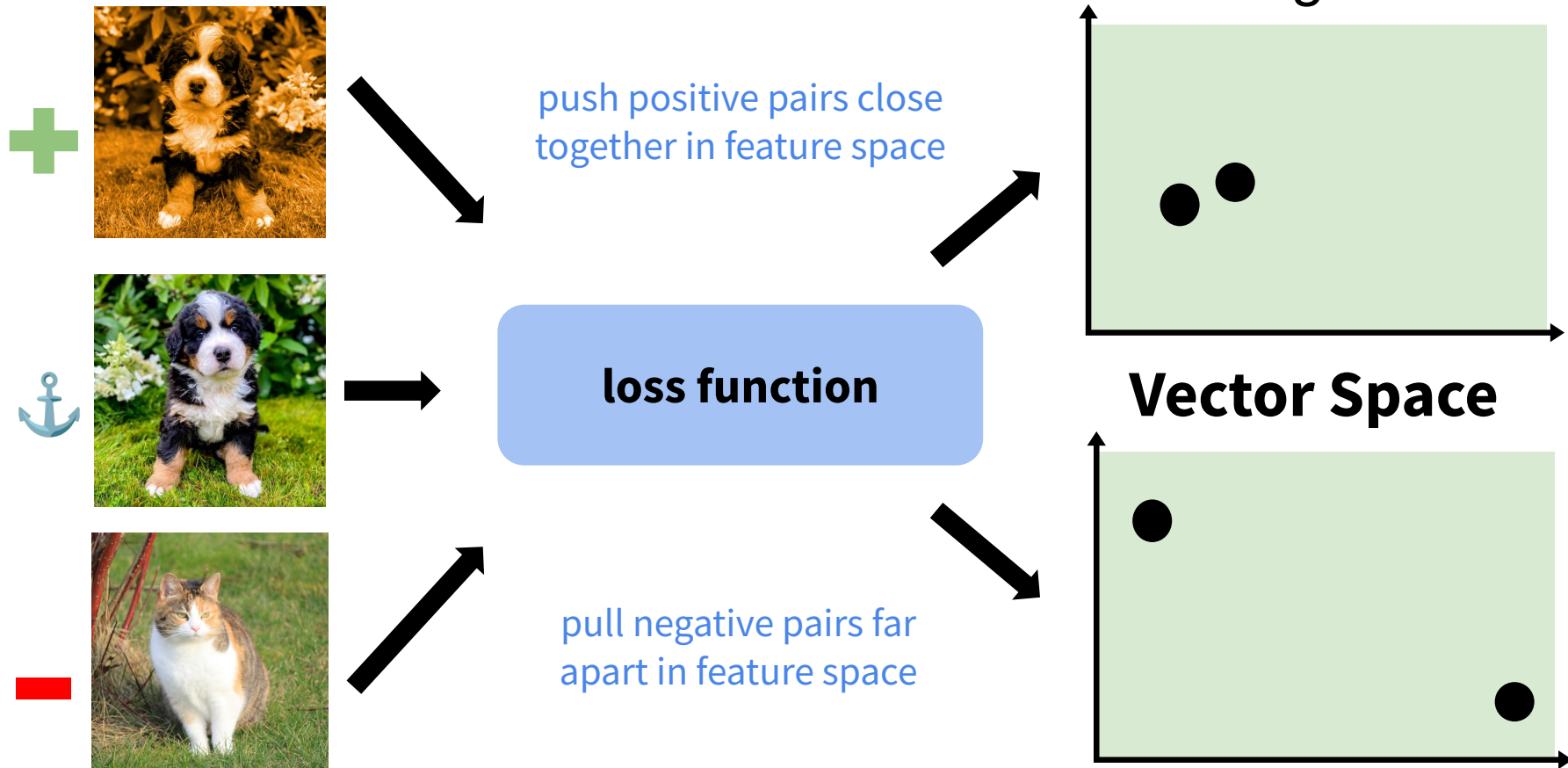
$f(x)$ = contrastive learning

Vector Space

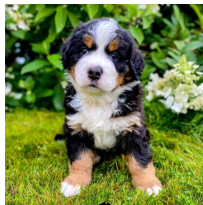


Any other image is a negative pair

Basic Model for Contrastive Learning



Triplet loss function



Anchor
example

$$\ell = \max(0, \|f(\mathbf{x}_i) - f(\mathbf{x}^+)\|^2 - \|f(\mathbf{x}_i) - f(\mathbf{x}^-)\|^2 + c)$$

Model



Positive pair



Negative pair

Triplet loss function

$$\ell = \max(0, \underbrace{\|f(\mathbf{x}_i) - f(\mathbf{x}^+)\|^2}_{\text{Model should map positive examples close together}} - \underbrace{\|f(\mathbf{x}_i) - f(\mathbf{x}^-)\|^2}_{\text{Model should map negative examples far apart}} + c)$$

Ensures
loss is not
negative

Model should map
positive examples
close together

Model should map
negative examples far
apart

Margin

Discuss

- Any potential problems with the triplet loss?
- Any ideas to remedy those problems

$$\ell = \max(0, \underbrace{\|f(\mathbf{x}_i) - f(\mathbf{x}^+)\|^2}_{\text{Model should map positive examples close together}} - \underbrace{\|f(\mathbf{x}_i) - f(\mathbf{x}^-)\|^2}_{\text{Model should map negative examples far apart}} + c)$$

Ensures
loss is not
negative

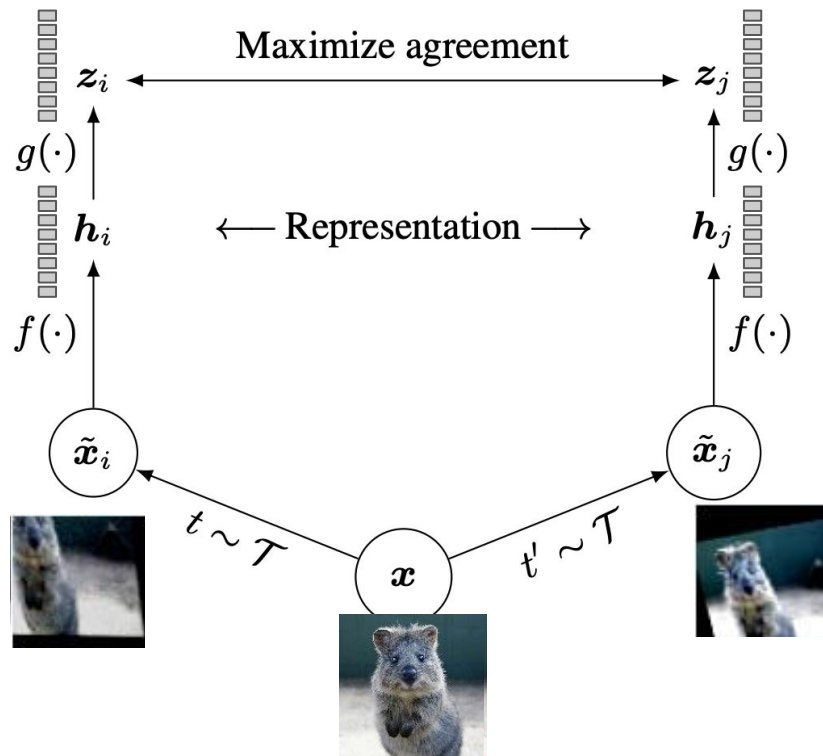
Model should map
positive examples
close together

Model should map
negative examples far
apart

Margin

SimCLR: A Simple Contrastive Learning Framework for Images

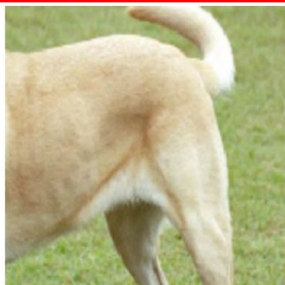
- Sample two different augmentations of an image
- Apply a base encoder to each view of the image to extract an image feature
 - e.g. ResNet
- Apply an MLP projection head to generate final representations
 - Throw away projection head after training



SimCLR Augmentations



(a) Original



(b) Crop and resize



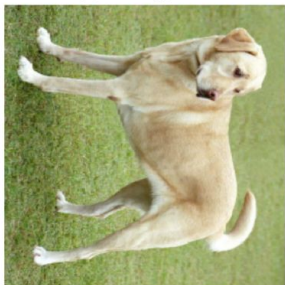
(c) Crop, resize (and flip)



(d) Color distort. (drop)



(e) Color distort. (jitter)



(f) Rotate $\{90^\circ, 180^\circ, 270^\circ\}$



(g) Cutout



(h) Gaussian noise



(i) Gaussian blur



(j) Sobel filtering

SimCLR Loss

- Temperature-scaled cross-entropy loss

Model should map positive
examples close together

$$l = -\log \left(\frac{\overbrace{\exp(\text{sim}(\mathbf{x}, \mathbf{x}^+)/\tau)}^{\text{Model should map positive examples close together}}}{\underbrace{\exp(\text{sim}(\mathbf{x}, \mathbf{x}^+)/\tau) + \exp(\text{sim}(\mathbf{x}, \mathbf{x}^-)/\tau)}_{\text{Model should map negative examples far apart}}} \right)$$

Model should map negative
examples far apart

Discussion: Comparison of Loss Functions

Consider when our model has learned to push most negative examples to have low similarity with the anchor. What happens to the value of each loss?

How does c affect Triplet Loss? (e.g. $c = 0.01$ vs. $c = 1$)

How does τ affect the SimCLR loss? (e.g. $\tau = 0.01$ vs. $\tau = 1$)

Triplet:

$$l = \max(0, \text{sim}(\mathbf{x}, \mathbf{x}^-) - \text{sim}(\mathbf{x}, \mathbf{x}^+) + c)$$

SimCLR loss:

$$l = -\log \left(\frac{\exp(\text{sim}(\mathbf{x}, \mathbf{x}^+)/\tau)}{\exp(\text{sim}(\mathbf{x}, \mathbf{x}^+)/\tau) + \exp(\text{sim}(\mathbf{x}, \mathbf{x}^-)/\tau)} \right)$$

SimCLR Algorithm

- Use other images in the mini-batch as negatives
- L2 normalize representations
 - Use cosine similarity as the distance metric
- Compute temperature-scaled cross-entropy for all positive pairs

Algorithm 1 SimCLR's main learning algorithm.

input: batch size N , constant τ , structure of f , g , $\mathcal{T}$.
for sampled minibatch $\{\mathbf{x}_k\}_{k=1}^N$ **do**
 for all $k \in \{1, \dots, N\}$ **do**
 draw two augmentation functions $t \sim \mathcal{T}, t' \sim \mathcal{T}$
 # the first augmentation
 $\tilde{\mathbf{x}}_{2k-1} = t(\mathbf{x}_k)$
 $\mathbf{h}_{2k-1} = f(\tilde{\mathbf{x}}_{2k-1})$ # representation
 $\mathbf{z}_{2k-1} = g(\mathbf{h}_{2k-1})$ # projection
 # the second augmentation
 $\tilde{\mathbf{x}}_{2k} = t'(\mathbf{x}_k)$
 $\mathbf{h}_{2k} = f(\tilde{\mathbf{x}}_{2k})$ # representation
 $\mathbf{z}_{2k} = g(\mathbf{h}_{2k})$ # projection
 end for
 for all $i \in \{1, \dots, 2N\}$ and $j \in \{1, \dots, 2N\}$ **do**
 $s_{i,j} = \mathbf{z}_i^\top \mathbf{z}_j / (\|\mathbf{z}_i\| \|\mathbf{z}_j\|)$ # pairwise similarity
 end for
 define $\ell(i, j)$ **as** $\ell(i, j) = -\log \frac{\exp(s_{i,j}/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(s_{i,k}/\tau)}$
 $\mathcal{L} = \frac{1}{2N} \sum_{k=1}^N [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$
 update networks f and g to minimize $\mathcal{L}$
end for
return encoder network $f(\cdot)$, and throw away $g(\cdot)$

Comparison of Loss Functions

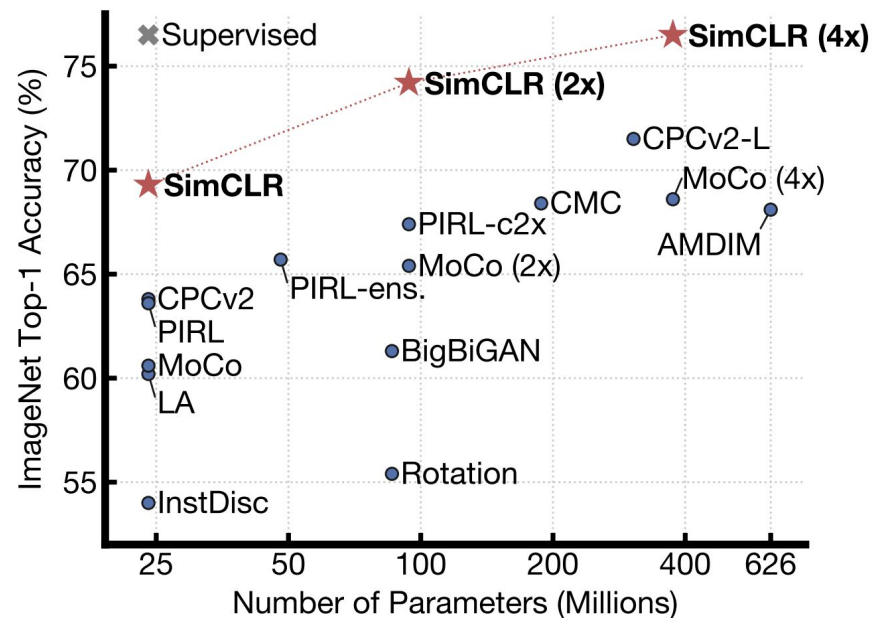
- Temperature-scaled cross entropy places more weight on hard negatives
 - Don't need to mine hard negatives

Name	Negative loss function	Gradient w.r.t. $\mathbf{u}$
NT-Xent	$\mathbf{u}^T \mathbf{v}^+ / \tau - \log \sum_{\mathbf{v} \in \{\mathbf{v}^+, \mathbf{v}^-\}} \exp(\mathbf{u}^T \mathbf{v} / \tau)$	$(1 - \frac{\exp(\mathbf{u}^T \mathbf{v}^+ / \tau)}{Z(\mathbf{u})}) / \tau \mathbf{v}^+ - \sum_{\mathbf{v}^-} \frac{\exp(\mathbf{u}^T \mathbf{v}^- / \tau)}{Z(\mathbf{u})} / \tau \mathbf{v}^-$
NT-Logistic	$\log \sigma(\mathbf{u}^T \mathbf{v}^+ / \tau) + \log \sigma(-\mathbf{u}^T \mathbf{v}^- / \tau)$	$(\sigma(-\mathbf{u}^T \mathbf{v}^+ / \tau)) / \tau \mathbf{v}^+ - \sigma(\mathbf{u}^T \mathbf{v}^- / \tau) / \tau \mathbf{v}^-$
Margin Triplet	$-\max(\mathbf{u}^T \mathbf{v}^- - \mathbf{u}^T \mathbf{v}^+ + m, 0)$	$\mathbf{v}^+ - \mathbf{v}^-$ if $\mathbf{u}^T \mathbf{v}^+ - \mathbf{u}^T \mathbf{v}^- < m$ else $\mathbf{0}$

Table 2. Negative loss functions and their gradients. All input vectors, i.e. $\mathbf{u}$, $\mathbf{v}^+$, $\mathbf{v}^-$, are ℓ_2 normalized. NT-Xent is an abbreviation for “Normalized Temperature-scaled Cross Entropy”. Different loss functions impose different weightings of positive and negative examples.

SimCLR Results

- Train a linear classifier on features from SimCLR
- Approaches supervised performance!



SimCLR Results

- Self-supervised vs. supervised ImageNet pre-training
- Evaluate transfer performance across 12 downstream classification datasets
 - Often outperforms supervised pre-training!

	Food	CIFAR10	CIFAR100	Birdsnap	SUN397	Cars	Aircraft	VOC2007	DTD	Pets	Caltech-101	Flowers
<i>Linear evaluation:</i>												
SimCLR (ours)	76.9	95.3	80.2	48.4	65.9	60.0	61.2	84.2	78.9	89.2	93.9	95.0
Supervised	75.2	95.7	81.2	56.4	64.9	68.8	63.8	83.8	78.7	92.3	94.1	94.2
<i>Fine-tuned:</i>												
SimCLR (ours)	89.4	98.6	89.0	78.2	68.1	92.1	87.0	86.6	77.8	92.1	94.1	97.6
Supervised	88.7	98.3	88.7	77.8	67.0	91.4	88.0	86.5	78.8	93.2	94.2	98.0
Random init	88.3	96.0	81.9	77.0	53.7	91.3	84.8	69.4	64.1	82.7	72.5	92.5

Effect Of Projection Head

- Projects data to “augmentation-invariant” representation

- Less useful features for downstream tasks

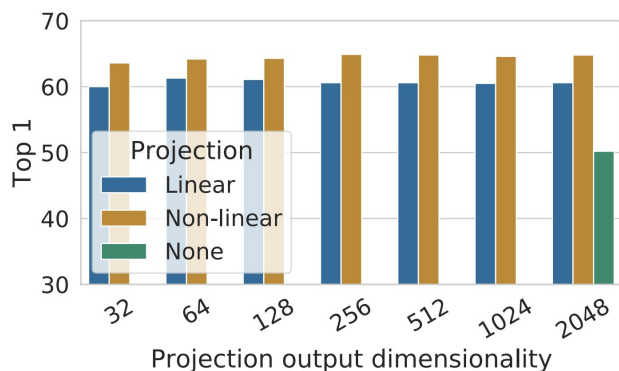
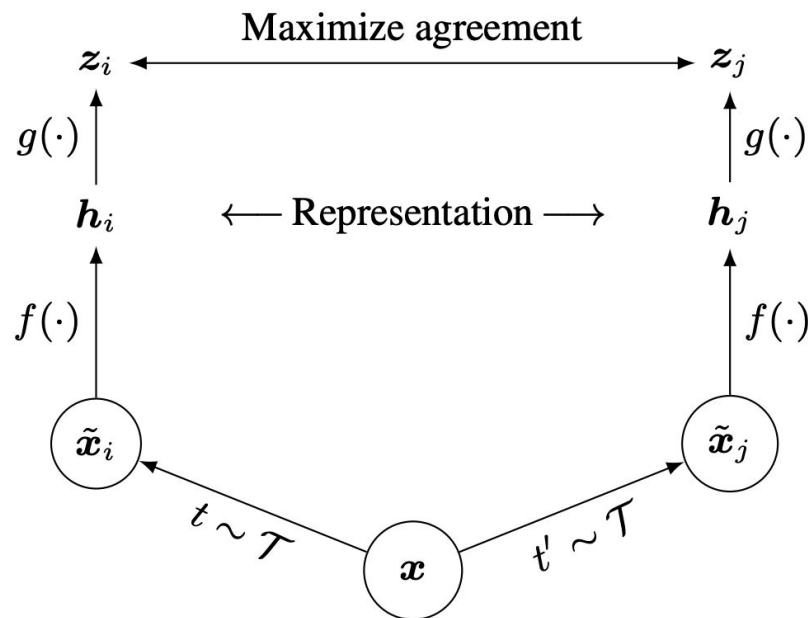
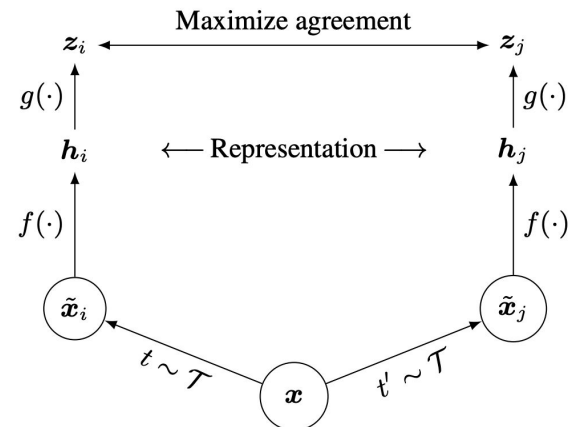
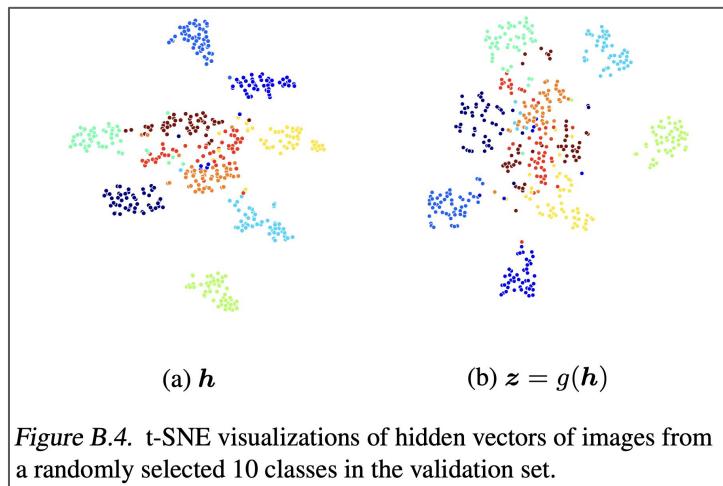


Figure 8. Linear evaluation of representations with different projection heads $g(\cdot)$ and various dimensions of $z = g(h)$. The representation h (before projection) is 2048-dimensional here.



Effect Of Projection Head

- Projects data to “augmentation-invariant” representation
 - Features less useful for downstream tasks



What to predict?	Random guess	Representation	
		h	$g(h)$
Color vs grayscale	80	99.3	97.4
Rotation	25	67.6	25.6
Orig. vs corrupted	50	99.5	59.6
Orig. vs Sobel filtered	50	96.6	56.3

Impact of Loss Function

- Proposed loss outperforms the margin loss
 - Even with negative mining
- L2 normalization is useful
- Sensitive to cross-entropy temperature

Margin	NT-Logi.	Margin (sh)	NT-Logi.(sh)	NT-Xent
50.9	51.6	57.5	57.9	63.9

Table 4. Linear evaluation (top-1) for models trained with different loss functions. “sh” means using semi-hard negative mining.

ℓ_2 norm?	τ	Entropy	Contrastive acc.	Top 1
Yes	0.05	1.0	90.5	59.7
	0.1	4.5	87.8	64.4
	0.5	8.2	68.2	60.7
	1	8.3	59.1	58.0
No	10	0.5	91.7	57.2
	100	0.5	92.1	57.0

Table 5. Linear evaluation for models trained with different choices of ℓ_2 norm and temperature τ for NT-Xent loss. The contrastive distribution is over 4096 examples.

Impact of Batch Size

- Requires large batches
 - Harder negatives!

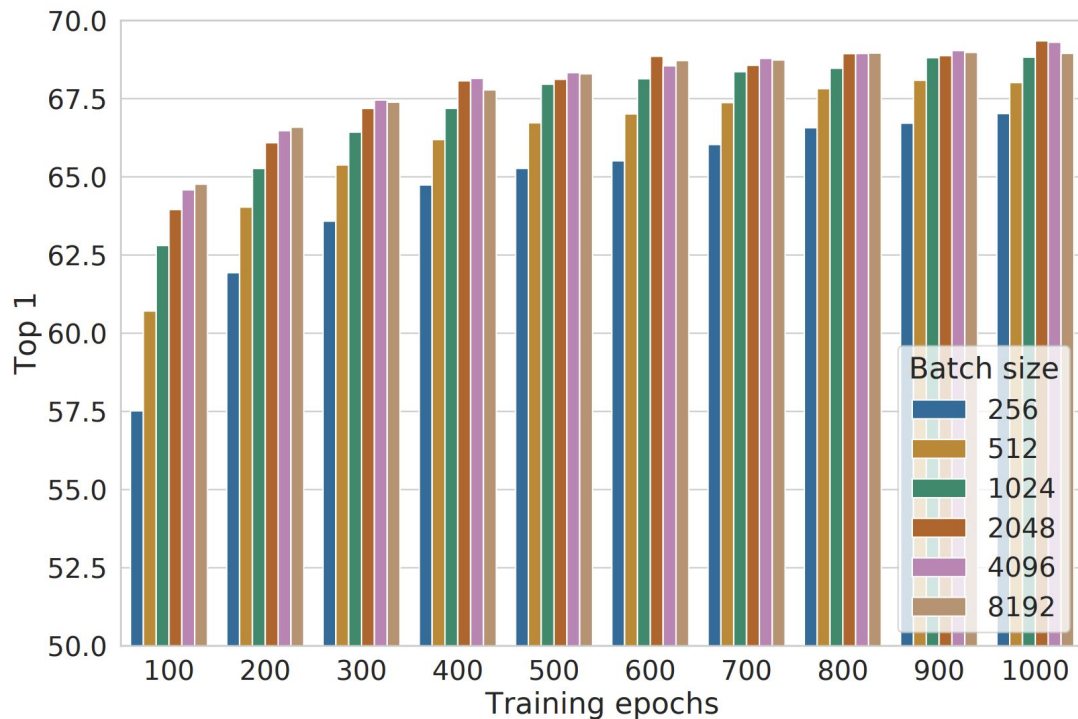
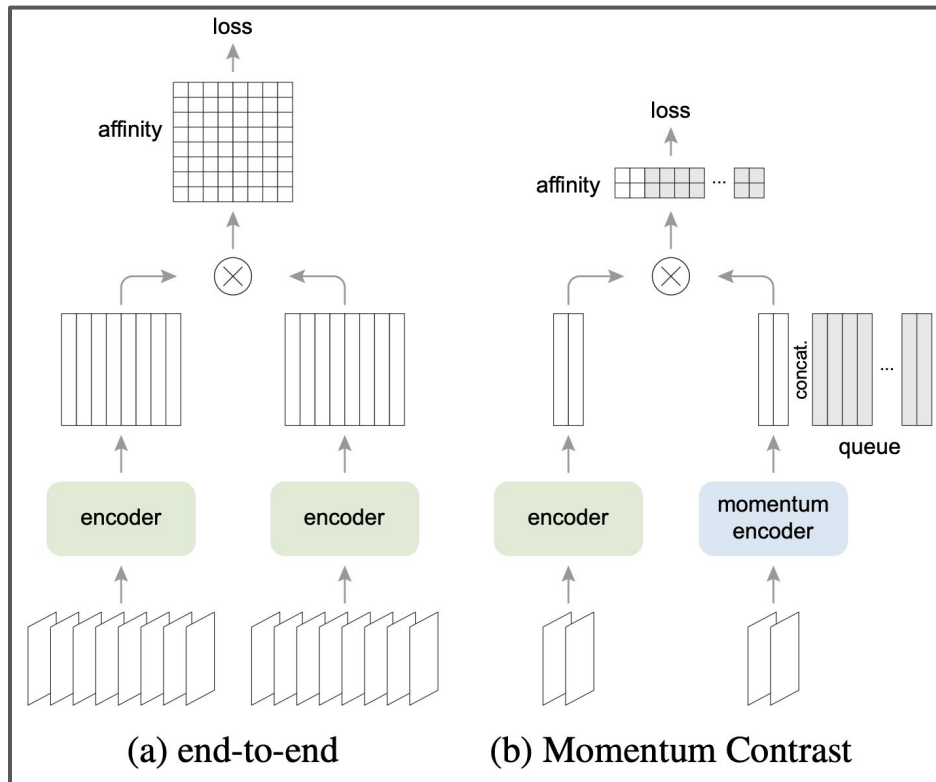


Figure 9. Linear evaluation models (ResNet-50) trained with different batch size and epochs. Each bar is a single run from scratch.¹⁰

Momentum Contrast (MoCo)

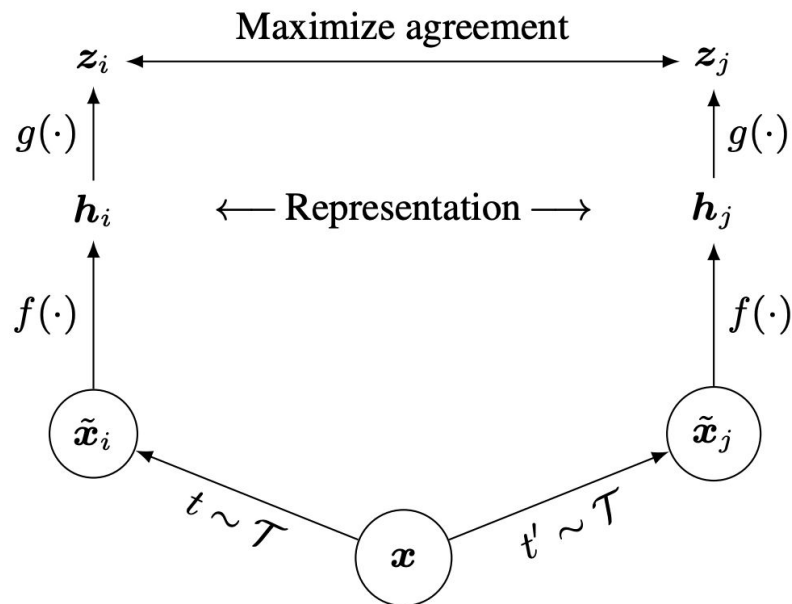
- Cache negative samples from earlier batches as you train
- Replace one encoder with an exponential moving average (EMA) of the model
 - Makes queued representations more stable

$$\theta_k \leftarrow m\theta_k + (1 - m)\theta_q$$



SimCLR: A Simple Contrastive Learning Framework for Images

- Sample two different augmentations of an image
- Apply a base encoder to each view of the image to extract an image feature
 - e.g. ResNet
- Apply an MLP projection head to generate final representations
 - Throw away projection head after training



MoCo v2

- MoCo v2: MoCo with some tricks from SimCLR
 - Stronger augmentations
 - MLP projection head
- Outperform SimCLR with modest batch sizes
 - Large numbers of negatives available from the queue

case	unsup. pre-train					ImageNet acc.
	MLP	aug+	cos	epochs	batch	
MoCo v1 [6]				200	256	60.6
SimCLR [2]	✓	✓	✓	200	256	61.9
SimCLR [2]	✓	✓	✓	200	8192	66.6
MoCo v2	✓	✓	✓	200	256	67.5
<i>results of longer unsupervised training follow:</i>						
SimCLR [2]	✓	✓	✓	1000	4096	69.3
MoCo v2	✓	✓	✓	800	256	71.1

Table 2. **MoCo vs. SimCLR**: ImageNet linear classifier accuracy (**ResNet-50, 1-crop 224×224**), trained on features from unsupervised pre-training. “aug+” in SimCLR includes blur and stronger color distortion. SimCLR ablations are from Fig. 9 in [2] (we thank the authors for providing the numerical results).

mechanism	batch	memory / GPU	time / 200-ep.
MoCo	256	5.0G	53 hrs
end-to-end	256	7.4G	65 hrs
end-to-end	4096	93.0G [†]	n/a

Table 3. **Memory and time cost** in 8 V100 16G GPUs, implemented in PyTorch. [†]: based on our estimation.

Retrieval Demo

<https://huggingface.co/spaces/vivien/clip>

Recap

- Supervised image classification pre-training produces strong image representations
 - Can efficiently transfer to other tasks
- Can apply self-supervised learning to images
 - Prefix tasks: rotation prediction, masked-image modeling, etc.
- Contrastive learning explicitly enforces similarity in representation space
 - Requires defining image augmentations